

CLASE 10 · MÓDULO 2

CI/CD y Pipelines de Calidad

Práctica — Guía para el alumno

Curso: Testing y Calidad de Software 2026

Duración: 90 minutos (más 5 min de setup)

Proyecto: TaskFlow (monorepo, apps/api)

Entregable: parte del Hito 4 — pipeline funcionando, bugs corregidos, badge en README

Al finalizar, podrás:

- Leer críticamente un workflow de GitHub Actions real e identificar sus piezas (jobs, triggers, needs, services).
- Modificar un workflow existente sin romper su funcionamiento.
- Configurar un threshold de cobertura como quality gate efectivo.
- Diagnosticar fallas en el pipeline leyendo el output de los tests, identificar el código defectuoso y corregirlo.
- Conectar el flujo de defensa en profundidad: pre-commit → CI → branch protection.

Setup (5 min)

Antes de empezar, verificá que tenés todo lo necesario. Si algo falla en esta sección, avisá al instructor antes de avanzar.

Prerequisitos

- Node 20 instalado (verificar con `node --version`).
- Repositorio TaskFlow clonado y dependencias instaladas (`npm ci` en el root).
- Cuenta GitHub con acceso de escritura al repositorio del equipo.
- Cuenta Codecov vinculada al repositorio (ver troubleshooting si falla).

Verificación rápida

Desde la raíz del monorepo, los tests deben pasar localmente:

```
node --version    # debe imprimir v20.x.x
npm ci            # instala dependencias (puede tardar 1-2 min)
cd apps/api
npm run test:unit # los tests unitarios deben correr (algunos fallarán: ¡eso es esperado!)
```

Nota sobre tests que fallan

Si al correr `npm run test:unit` ves que 4 tests fallan, NO es un problema de tu instalación. Son los bugs latentes que vamos a diagnosticar en la Parte C. Anotalo y seguí adelante.

Parte A — Lectura crítica del pipeline existente (30 min)

TaskFlow ya tiene un workflow de CI configurado en `.github/workflows/ci.yml`. Antes de modificarlo, vamos a entenderlo completo. En la industria, esto es lo más común: rara vez construís un pipeline desde cero — heredás uno y tenés que aprender a leerlo.

A.1 — Primer contacto (5 min)

1. Abrí `.github/workflows/ci.yml` en tu editor.
2. Leelo completo, de arriba a abajo, sin saltar líneas.
3. Marcá en una hoja (o como comentarios) las dudas que tengas: palabras que no entendés, sintaxis confusa, líneas que no sabés para qué están.
4. No las resuelvas todavía. Llevalas al ejercicio de la siguiente sección.

Tip de lectura

Empezá identificando las tres claves obligatorias que vimos en teoría: `name:`, `on:` y `jobs:`. Después contá cuántos jobs hay y anotá sus nombres en orden. Eso te da el esqueleto del pipeline.

A.2 — Ejercicio (10 min)

Completen los siguientes dos ejercicios. Pueden volver a abrir el `ci.yml` todas las veces que necesiten.

Ejercicio 1: Diagrama del grafo de jobs

Dibujá (en papel o pizarra) las dependencias entre los 5 jobs. Usá flechas para indicar qué job depende de cuál. Pregunta guía: si reviso las cláusulas `needs:` de cada job, ¿qué grafo se forma?

```
lint > unit-tests > integration-tests > bdd-tests > e2e-tests
```

Indicá también:

- Qué jobs corren en paralelo (si los hay). No hay ya que el trabajo siguiente depende del anterior.
- Cuál es el job más temprano del flujo. El lint.
- Cuál es el último. E2E tests con Playwright
- Si algún job tiene condiciones especiales para ejecutarse (cláusula `if:`). `e2e-tests` tiene `if: github.event_name == 'pull_request'`, por lo que solo se ejecuta cuando se abre un PR contra main.

Ejercicio 2: 5 preguntas guiadas

Respondé más abajo (no hace falta que sean respuestas largas, una o dos oraciones alcanzan):

1. ¿Por qué el job `e2e-tests` tiene `if: github.event_name == 'pull_request'`? ¿Qué efecto tiene esa línea? Está porque los e2e son muy pesados y frágiles entonces solo se ejecutan cuando va a haber un merge con main.

- ¿Qué pasaría si elimino needs: lint del job unit-tests? ¿Por qué se incluye esa dependencia? Ahí unit-tests y lint se ejecutarían en paralelo.
- ¿Por qué los jobs de integration-tests y bdd-tests definen un service: postgres pero el job unit-tests no? ¿Qué hace ese bloque services:? Porque los tests unitarios utilizan mocks y no un postgres real.
- En el job e2e-tests hay un step que usa upload-artifact con if: failure(). ¿Qué propósito cumple? ¿Cuándo se ejecuta? Sirve para empaquetar y subir el reporte de Playwright y solo se ejecuta cuando falla para saber que salió mal.
- ¿Por qué la línea cache: 'npm' aparece en todos los jobs? ¿Qué pasa si la quito? Para que guarde las dependencias del package-lock.json, sino se descargan en cada ejecución.

Respuestas:

A.3 — Puesta en común (10 min)

Workflows complementarios: nightly.yml

Mientras revisan el ci.yml, abran también .github/workflows/nightly.yml. Este es un workflow distinto que corre sobre un trigger diferente al de ci.yml.

Preguntas para la discusión grupal:

- ¿Cuál es el trigger de este workflow? ¿Qué significa la sintaxis del cron? El trigger es schedule escrito en cron como: 0 2 * * *, que significa que se va a ejecutar todos los días a las 2am.
- ¿Por qué los tests de performance no están en ci.yml junto con los otros? Porque estos cuestan mucho tiempo de ejecutar y requieren de un entorno estable que no se puede conseguir en los CI.
- ¿Qué ventaja tiene separar workflows según su frecuencia/costo de ejecución? Porque así se pueden ver los errores por capas, primero se ve si se rompió algo de funcionamiento básico en los tests unitarios y después se va ampliando el scope para abarcar todo el sistema

A.4 — Pequeña modificación práctica (5 min)

Vas a agregar la opción de ejecutar el workflow manualmente. Esto es útil para hotfixes, debugging, o re-correr un pipeline sin necesidad de hacer un commit nuevo.

- Abrí ci.yml.
- Ubicá el bloque on: al principio del archivo.
- Agregá una línea workflow_dispatch: dentro de on: (al mismo nivel que push: y pull_request:).

El bloque on: debería quedar así:

```
on:
  push:
    branches: [ '*' ]
```

```
pull_request:
  branches: [main]
workflow_dispatch: # ← línea nueva
```

1. Commiteá con un mensaje convencional: `git commit -m "ci: agregar dispatch manual al workflow"`
2. Pusheá: `git push`
3. Andá a la pestaña Actions del repositorio en GitHub.
4. Seleccioná el workflow "CI — TaskFlow" en la barra izquierda.
5. Hací click en el botón "Run workflow" (arriba a la derecha) y luego en "Run workflow" en el menú desplegable.
6. Verificá que aparece un nuevo run, marcado como manualmente disparado.

Qué acabás de aprender

`workflow_dispatch` convierte cualquier workflow en ejecutable a demanda. Esto es defensa en profundidad operativa: te da control manual cuando lo necesitás (debugging, re-runs después de un servicio externo caído, etc.) sin depender del flujo automático de push/PR.

Parte B — Profundizar quality gates (20 min)

El pipeline ya corre los tests y sube cobertura a Codecov, pero todavía no tiene un gate que bloquee builds con cobertura insuficiente. En esta parte vas a agregar ese gate en la herramienta correcta: Vitest.

B.1 — Inspeccionar la config actual de coverage (5 min)

1. Abrió `apps/api/vitest.config.ts`.
2. Identificá la sección `test.coverage`. Tomá nota de:
 - Qué provider usa (v8, istanbul, etc.)
 - Qué reporters genera (text, html, json, lcov, etc.)
 - Qué archivos excluye de la medición (carpeta exclude)
3. Abrió `package.json` (en `apps/api/`) y revisá el script `test:unit`.
4. Verificá que el script efectivamente genera el reporte de cobertura cuando se ejecuta en CI (debería incluir `--coverage` o estar configurado para que Vitest lo haga por defecto).

B.2 — Agregar threshold de cobertura (10 min)

Vitest soporta thresholds de cobertura nativamente. Si la cobertura cae por debajo del umbral configurado, el comando de tests termina con código de error y el job del pipeline falla.

Agregar el threshold

En `apps/api/vitest.config.ts`, dentro de `test.coverage`, agregá el bloque `thresholds`:

```
coverage: {
  provider: 'v8',
  reporter: ['text', 'json', 'lcov'],
  // ... el resto de tu config
  thresholds: {
    lines: 80,
    branches: 75,
    functions: 80,
    statements: 80,
  },
},
```

Probarlo localmente

Ejecutá:

```
cd apps/api
npm run test:unit -- --coverage
```

Si la cobertura actual está por encima de los thresholds, el comando termina exitosamente. Vas a ver una tabla con los porcentajes por archivo.

Provocar un fallo intencional

Para ver qué pasa cuando el gate falla, subí temporalmente el threshold de líneas a 95%:

```
thresholds: {
  lines: 95,    // ← subimos a propósito
  branches: 75,
```

```
functions: 80,
statements: 80,
},
```

Volvé a correr `npm run test:unit -- --coverage`. Deberías ver un error como:

```
ERROR: Coverage for lines (XX.XX%) does not meet global threshold (95%)
```

Esto es lo que el pipeline va a hacer también

Cuando este threshold no se cumple, el proceso termina con exit code distinto de 0. El job del workflow detecta eso y marca el run como failed. El PR no se puede mergear hasta que la cobertura suba.

Restaurar y commitear

1. Volvé lines: 95 a lines: 80.
2. Verificá que `npm run test:unit -- --coverage` pasa de nuevo.
3. Commiteá: `git commit -m "feat(api): agregar threshold de coverage como quality gate"`
4. Pusheá y observá el pipeline corriendo con el nuevo gate.

B.3 — Verificar Codecov (5 min)

Codecov ya está integrado en el pipeline (vimos el paso en `ci.yml`). Vamos a confirmar que funciona y entender qué información agrega.

1. Entrá a <https://codecov.io/> y authenticate con tu cuenta de GitHub.
2. Buscá el repositorio de TaskFlow.
3. Identificá en el dashboard:
 - El porcentaje global de cobertura del proyecto.
 - Los archivos con menor cobertura (oportunidades de mejora).
 - La evolución de la cobertura a lo largo del tiempo (gráfico histórico).
4. Si tu equipo tiene algún PR abierto, revisá el comentario automático que Codecov agrega: muestra el delta de cobertura (cuánto sube o baja con ese PR).

Codecov vs threshold local: ¿no es lo mismo?

No exactamente. El threshold de Vitest es un gate técnico: si no se cumple, el pipeline falla. Codecov es información visual y comentarios automáticos en PRs: ayuda a entender la evolución y los detalles archivo por archivo, pero no bloquea por sí solo (a menos que configures branch protection rules conectadas).

Parte C — Bugs latentes (40 min)

El framing

Su pipeline está terminado y funciona. Pero TaskFlow es un proyecto real con código real, y como todo proyecto real, tiene bugs latentes que nadie había detectado.

Su pipeline va a hacer lo que hace todo buen pipeline el día que se enchufa por primera vez: encontrar los bugs que estaban ahí desde siempre.

No vamos a plantar nada. Los bugs ya existen en main desde que se escribió el código de TaskFlow. Lo que cambia hoy es que ahora tenés un pipeline funcionando que los va a hacer visibles.

C.1 — Primer run completo (5 min)

1. Asegurate de estar en una rama de trabajo (no main).
2. Pusheá los cambios que hiciste en la Parte B.
3. Andá a la pestaña Actions de GitHub.
4. Observá el run completo. ¿En qué job falla? ¿Cuántos tests fallan?
5. Hacé click en el job que falló para ver el output detallado.

Anotá lo que ves:

- Job en el que falla: _____
- Cantidad de tests fallidos: _____
- Archivos de test involucrados: _____

C.2 — Análisis grupal del test report (5 min)

Cuando varios tests fallan a la vez, no es lo mismo atacarlos en orden aleatorio que en el orden correcto. Vamos a aplicar un principio simple:

Principio: atacá el más informativo primero

Un test que dice exactamente qué string esperaba y qué string recibió (un diff literal) es más fácil de arreglar que uno que dice "la promesa resolvió cuando se esperaba que rechazara". Empezá por los fáciles: cada bug que arreglás puede revelar información sobre los otros, y vas ganando confianza para los difíciles.

En grupo (toda la clase), revisá los outputs de los 4 tests fallidos:

- ¿Cuál de los 4 outputs es el más informativo?
- ¿Cuál parece el más simple de arreglar?
- ¿Hay tests que parecen estar relacionados con el mismo bug subyacente?

Spoiler para que no se pierdan: hay 2 bugs distintos, y cada uno hace fallar 2 tests. Vamos a atacar primero el más simple (lo llamaremos BUG-04) y después el más complejo (BUG-03).

C.3 — Diagnosticar y arreglar BUG-04 (10 min)

Los 2 tests más informativos son los que muestran diff literal de strings:

```
X rechaza contraseña menor a 8 caracteres
  AssertionError: expected [Function] to throw error including
  'at least 8 characters' but got:
  '[{"code":"too_small","message":"La contraseña debe tener
    al menos 8 caracteres","path":["password"]}]'

X lanza ConflictError si el email ya existe
  AssertionError: expected [Function] to throw error including
  'Email already registered' but got 'Email ya registrado'
```

Diagnóstico

Leé los outputs. ¿Qué patrón notás?

- Los tests esperan los mensajes de error en inglés.
- El código actual los devuelve en español.

El bug no es un bug funcional (la validación rechaza la contraseña corta, eso funciona). El bug es que los strings esperados están en idioma distinto al que devuelve el código. En este caso, los tests son la fuente de verdad: definen qué espera ver un consumidor de la API.

Reparación

1. Buscá el string 'Email ya registrado' en el repo (Cmd/Ctrl + Shift + F en VS Code).
2. Lo vas a encontrar en apps/api/src/services/auth.service.ts línea 37.
3. Cambialo a 'Email already registered'.
4. Buscá ahora 'La contraseña debe tener al menos 8 caracteres'.
5. Lo vas a encontrar en apps/api/src/services/auth.service.ts línea 14.
6. Cambialo a 'Password must be at least 8 characters'.

Verificar localmente

```
cd apps/api
npm run test:unit
```

Deberías ver 2 tests menos fallando (de 4 a 2). Si todavía fallan los 4, releé el output: probablemente cambiaste un string pero no el otro.

Commit y push

```
git add apps/api/src/services/auth.service.ts
git commit -m "fix(auth): unificar mensajes de error en inglés"
git push
```

Andá a Actions y verificá que el pipeline avanzó (todavía rojo, pero con menos tests fallados que antes).

C.4 — Re-correr y discusión (5 min)

Punto de discusión

Corregir BUG-04 hizo que el output del pipeline sea más limpio: ahora solo quedan 2 fallos en vez de 4. ¿Por qué importa esto?

Cuando un test falla, su error contamina el output. A veces dos bugs distintos parecen el mismo, o un bug oculta otro. Corregir los fáciles primero NO es "hacer trampa": es una estrategia de diagnóstico legítima que reduce el ruido y deja visibles los bugs reales.

Antes de seguir, en pareja respondan:

- ¿En qué se parecen los 2 tests que quedan fallando?
- ¿Qué tienen en común sus mensajes de error?
- ¿Qué está testeando cada uno (en términos de la lógica de negocio)?

C.5 — Diagnosticar y arreglar BUG-03 (10 min)

Los 2 tests restantes muestran un comportamiento curioso:

```
X rechaza contraseña sin mayúscula
  AssertionError: promise resolved "{ user: {...}, token: "eyJ..." }"
  instead of rejecting
  > auth.service.spec.ts:72:7 → .rejects.toThrow('uppercase')

X rechaza contraseña sin número
  AssertionError: promise resolved "{ user: {...}, token: "eyJ..." }"
  instead of rejecting
  > auth.service.spec.ts:78:7 → .rejects.toThrow('number')
```

Diagnóstico

Estos outputs son distintos: no comparan strings, sino comportamientos. Decoded:

- Los tests llaman a register(...) con una contraseña que NO debería ser válida.
- Esperan que la llamada falle (rejects.toThrow).
- Pero en lugar de fallar, la llamada resuelve exitosamente con un token de autenticación.

Conclusión: la validación está incompleta. La función register() acepta contraseñas que los tests consideran inválidas. El bug no es un mensaje mal puesto: es una regla de validación que falta.

Investigar el schema

1. Abrí apps/api/src/services/auth.service.ts.
2. Buscá el RegisterSchema cerca del inicio del archivo (líneas 10-20).
3. Mirá las validaciones de password actuales:

```
password: z
  .string()
  .min(8, 'Password must be at least 8 characters'),
  // ← acá faltan reglas
```

Las únicas reglas son: debe ser string y debe tener al menos 8 caracteres. Pero los tests esperan que también se valide:

- Que contenga al menos una letra mayúscula (busca 'uppercase' en el mensaje).

- Que contenga al menos un número (busca 'number' en el mensaje).

Reparación

Zod permite encadenar validaciones con `.regex()`. Agregá las dos reglas faltantes:

```
password: z
  .string()
  .min(8, 'Password must be at least 8 characters')
  .regex(/[A-Z]/, 'Password must contain an uppercase letter')
  .regex(/[0-9]/, 'Password must contain a number'),
```

Verificar y pushear

```
cd apps/api
npm run test:unit          # debería pasar todos los tests ahora
git add apps/api/src/services/auth.service.ts
git commit -m "fix(auth): agregar validaciones de complejidad en password"
git push
```

Andá a Actions. Esperá a que el pipeline corra completo. Deberías ver:

- Job lint: ✓ verde
- Job unit-tests: ✓ verde (¡todos los tests pasan!)
- Jobs integration-tests, bdd-tests: ✓ verde
- Job e2e-tests: se ejecuta solo si abriste un PR; si estás en push directo, se saltea

C.6 — Cierre (5 min)

Lo que acabás de hacer

1. Levantaste un pipeline funcional sobre código real.
2. El pipeline detectó automáticamente 2 bugs latentes en TaskFlow.
3. Diagnosticaste cada uno leyendo el output del pipeline (no a ciegas).
4. Aplicaste el principio de "atacar el más informativo primero" para reducir ruido.
5. Corregiste los bugs y dejaste el pipeline en verde.

Esto es exactamente lo que pasa cuando un equipo adopta CI/CD por primera vez en un proyecto que venía sin él: el pipeline encuentra bugs que llevaban semanas o meses ocultos. No es magia; es shift-left funcionando como debe.

Discusión final

- Si tuvieras que documentar este flujo para un compañero nuevo, ¿qué pasos resaltarías?
- ¿Cómo conecta esta actividad con el concepto de "defensa en profundidad" que vimos en teoría?

Entregables del Hito 4

Al finalizar esta práctica, tu equipo debe tener:

1. Pipeline ejecutándose correctamente sobre la rama de trabajo, con todos los jobs en verde.
2. workflow_dispatch agregado al ci.yml (Parte A.4).
3. Threshold de coverage configurado en apps/api/vitest.config.ts (Parte B.2).
4. BUG-03 corregido: validaciones de password completas en RegisterSchema.
5. BUG-04 corregido: mensajes de error unificados en inglés.
6. Badge del pipeline en el README.md raíz del proyecto:

```

```

Troubleshooting

Errores comunes que pueden aparecer durante la práctica, ordenados por la parte donde es más probable que surjan.

Setup

npm ci falla con error de versión de Node

El campo engines en el package.json raíz pide Node 20.x. Si tenés otra versión:

Verificá con `node --version`. Si es < 20, instalá Node 20 con nvm:

```
nvm install 20 && nvm use 20
```

Los tests no corren localmente ("command not found: vitest")

Faltan dependencias. Asegurate de haber corrido npm ci en la raíz del monorepo, no solo en apps/api:

```
cd <raiz-monorepo> && npm ci
```

Parte A — Pipeline en GitHub Actions

El workflow no se dispara después de pushear

Verificá tres cosas:

- ¿La rama donde estás pusheando matchea con branches: ['**'] (que es "cualquier rama")? Si por error cambiaste a branches: [main], solo dispara en main.
- ¿El archivo está en .github/workflows/ exactamente? Tiene que ser ese path.
- ¿El YAML es válido? GitHub te muestra un warning en la pestaña Actions si hay un error de sintaxis.

El job de integration-tests falla con "connection refused" a Postgres

El service de Postgres tarda en arrancar. Normalmente el healthcheck cubre eso, pero a veces hay timing issues. Mirá el log:

- Si el job ni siquiera intentó conectar: el service no arrancó. Revisá la sintaxis del bloque services: en el job.
- Si la conexión falla intermitentemente: el healthcheck no está bien configurado. Asegurate de que las opciones --health-cmd y --health-retries están presentes.

Parte B — Coverage y Codecov

El test de coverage pasa localmente pero falla en CI

Causas más comunes:

- Diferencia de versión de Node: localmente usás Node 22, pero CI usa Node 20. Diferentes versiones cubren código distinto.
- Variables de entorno: CI no tiene un .env, usa las variables declaradas en env: del workflow. Si tu test depende de una variable que solo existe local, va a fallar.
- Archivos excluidos: tu config local puede tener más exclusiones que la que está en el repo.

Codecov no aparece en el PR

Pasos para diagnosticar:

- Verificá que el secreto CODECOV_TOKEN esté configurado en Settings → Secrets and variables → Actions del repo.
- El job unit-tests debe haber terminado OK. Si falla, no se sube el reporte.
- Revisá el step "Upload coverage report" en el log del job: debería decir "Uploaded coverage report" al final.
- Esperá unos segundos: Codecov a veces tarda 30-60 seg en procesar y comentar.

Parte C — Bugs latentes

Corregí BUG-04 pero los 4 tests siguen fallando

Casi seguro cambiaste un string pero olvidaste el otro. Buscá en el repo ambos strings originales: `'Email ya registrado'` y `'La contraseña debe tener al menos 8 caracteres'`. Los dos tienen que estar reemplazados.

Corregí BUG-03 pero los tests de password siguen fallando

Posibles causas:

- Las regex deben estar dentro del método `.regex(...)` de Zod, no en otro lugar.
- El mensaje de error debe contener exactamente las palabras `'uppercase'` y `'number'` (en inglés) — los tests buscan ese string.
- Las dos validaciones se encadenan con punto: `.min(8, ...).regex(...).regex(...)` — no se separan con coma.

Errores misceláneos

tsc --noEmit falla pero compila localmente

Tu IDE puede estar más permisivo que el tsc del pipeline. Revisá el tsconfig.json de apps/api: el pipeline usa exactamente ese archivo con la flag --project. Si tu IDE usa una config distinta (por ejemplo, una composite o un override), puede mostrar verde algo que el CLI rechaza.

El pipeline está muy lento (> 10 min)

Verificá:

- Que cache: 'npm' esté presente en todos los jobs (no solo el primero).
- Que el primer run del día tarde más es normal: el caché se construye en ese run. Los siguientes deberían ser ~50% más rápidos.
- Si nunca se hace más rápido: probablemente el package-lock.json se modificó. El caché se invalida cuando cambia el lock.